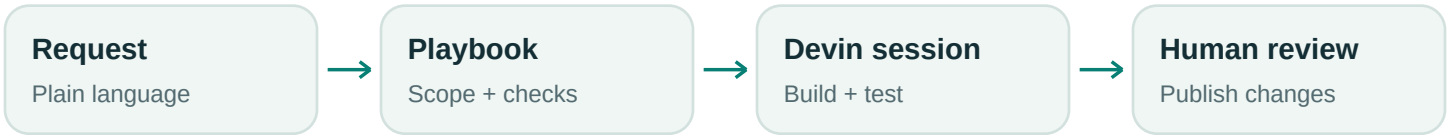
Make the next tool easier.

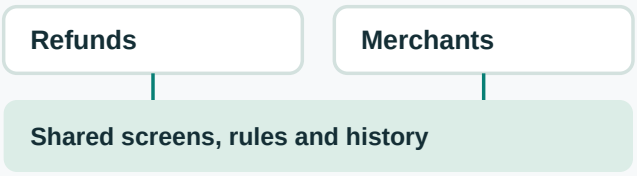
I tested the work after the first build: changing a refunds queue, reusing it for merchant reviews, and adding a separate settlement app with a custom integration.

01 A REPEATABLE WAY TO BUILD AND CHANGE



02 TWO SMALL PROOFS

Reuse a foundation



React + Python + SQLite; separate records

Own an integration

Settlement review

App backend → **Mock vendor**

Server-held credentials; scoped tokens. Local sign-in, roles and tenant checks.

03 THE CHOICES THAT MATTER

1

Test the second request, not just the first screen.

Refunds cover search, decisions and history. A fresh session added an overdue filter; merchant reviews reused the base. Shared code saves duplication, but changes can affect both apps.

2

Use code where the integration needs it.

Settlement uses OAuth client credentials on the backend. Power Apps custom connectors do not support that grant directly; an intermediary is possible. The benefit is control, with another service to maintain.

3

Separate app access from Devin access.

Settlement checks roles and tenants on the server. Devin has a restricted profile; the automation is configured for approved requesters. Playbooks guide work and tags organize it. Neither grants permission.

What this proves — and what it leaves open

Working changes, reuse and a simulated integration. Synthetic accounts and SQLite kept scope small. Production identity, protected audit history, hosting and backups remain. Automatic trigger filtering and profile enforcement need conclusive tests.

My next step: Keep Power Apps; pilot one new, low-risk tool. Measure review and support time before trading the license bill for engineering ownership.

Settlement follow-up timed separately. Tests, HTTP checks and browser journeys passed locally.

Devin was the primary builder; Codex assisted orchestration, review, fixes and presentation.

Source repository **Microsoft: connector limitation**